Introducción

En función de la altura, podemos decir lo siguiente:

Un árbol binario de búsqueda balanceado (AVL) es un árbol binario de búsqueda con una propiedad adicional de equilibrio, según la cual las alturas de los hijos derecho e izquierdo solo pueden diferir, a lo sumo, en una unidad. Como es usual, tomamos -1 como la altura del árbol vacío.

Cuando insertamos un nuevo nodo al árbol, el método implementado hasta ahora no nos asegura que el árbol permanezca balanceado y, por lo tanto, el grado de balance depende del orden en el que insertamos los nodos en el árbol.

Si sabemos que el árbol binario de búsqueda es balanceado, entonces podemos asegurar que encontrar un elemento en el árbol requiere, a lo sumo, O(log n).

Las operaciones básicas de un árbol AVL implican generalmente realizar los mismos algoritmos que serían efectuados en un árbol binario de búsqueda desequilibrado (desbalanceado), pero precedido o seguido por una o más de las llamadas *rotaciones AVL* (Weiss, 2000).

1. Rotaciones

El reequilibrado se produce de abajo hacia arriba sobre los nodos en los que se produce el desequilibrio. Pueden darse dos casos: rotación simple o rotación doble; a su vez, ambos casos pueden ser hacia la derecha o hacia la izquierda.

Rotación simple

El siguiente gráfico muestra la rotación simple. El árbol de la izquierda representa un instante antes del ajuste (en ese instante, el árbol se encuentra desbalanceado) y el árbol de la derecha muestra el árbol luego de realizar la rotación simple.

El siguiente código muestra las rotaciones simples implementadas en Java:

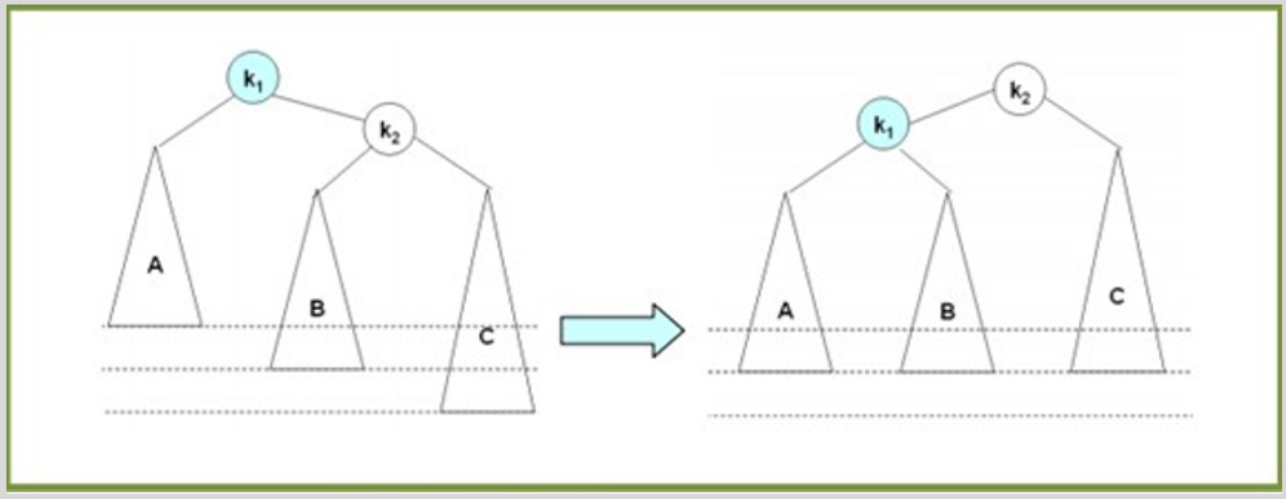
Figura 1: Código de rotaciones simples

```
private NodoBinario rotacionSimpleIzq(NodoBinario k2) {
    NodoBinario k1 = k2.getIzquierdo();
    k2.setIzquierdo(k1.getDerecho());
    k1.setDerecho(k2);
    return k1;
}

private NodoBinario rotacionSimpleDer(NodoBinario k1) {
    NodoBinario k2 = k1.getDerecho();
    k1.setDerecho(k2.getIzquierdo());
    k2.setIzquierdo(k1);
    return k2;
}
```

Fuente: elaboración propia.

Figura 2: Rotaciones simples



Fuente: elaboración propia.

El método conHijoIzq realiza la rotación de un nodo de un árbol binario con hijo izquierdo y el método conHijoDer realiza la rotación de un nodo de un árbol binario con hijo derecho.

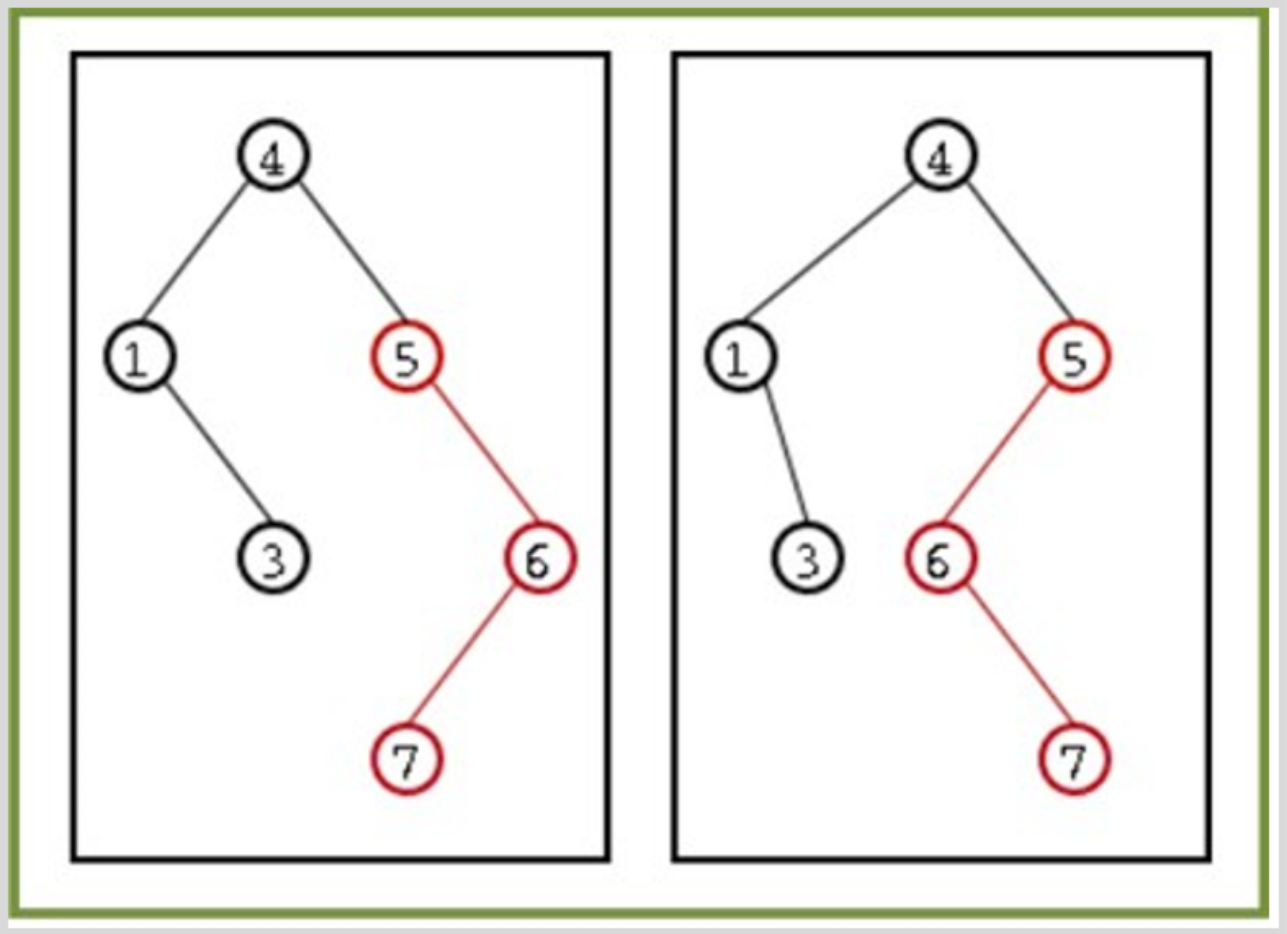
Rotación doble

Una rotación doble consiste simplemente en realizar dos rotaciones simples, ya que existen casos en que, utilizando una rotación simple, el árbol sigue quedando desbalanceado. En otras palabras:

Hemos visto cómo restaurar la propiedad de equilibrio cuando se presentan desequilibrios “hacia la izquierda” o “hacia la derecha” luego de realizar inserciones en un árbol AVL. Sin embargo y como veremos, pueden ocurrir “desequilibrios en zig-zag”, es decir, desequilibrios que no son ni a la derecha ni a la izquierda (Gurin, 2004).

Como es el caso de estos árboles:

Figura 3: Ejemplo de desequilibrio en zigzag



Fuente: elaboración propia.

Como podemos ver en la siguiente figura, una rotación simple no soluciona el problema de desequilibrio. En estos casos se aplica otro tipo de rotación, denominada *rotación doble*, la cual, análogamente a la rotación simple, puede ser izquierda o derecha según el caso.

En realidad, la rotación doble constará de dos rotaciones simples. El siguiente código muestra las rotaciones dobles implementadas en Java:

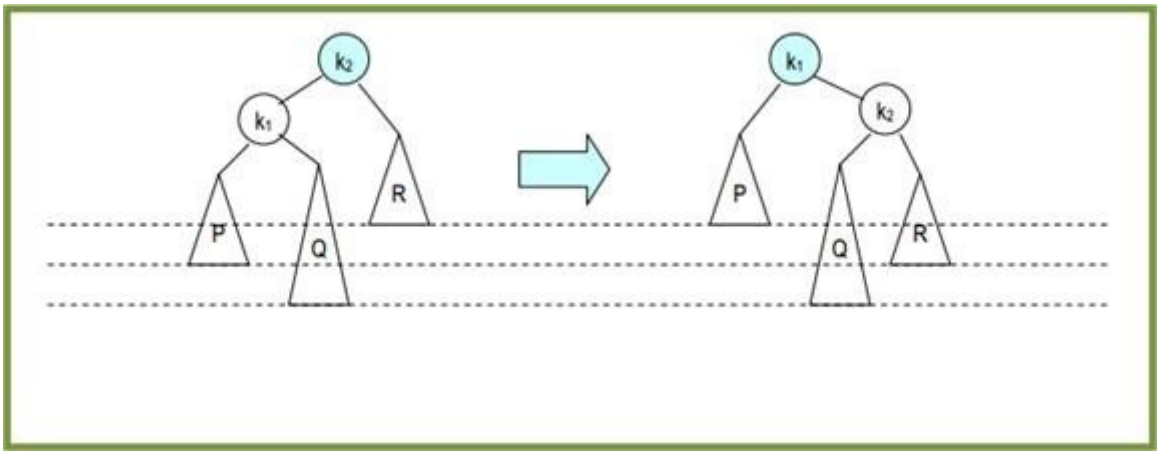
Figura 4: Código de rotaciones dobles

```
private NodoBinario rotacionDobleIzqDer(NodoBinario k3) {
    k3.setIzquierdo(rotacionSimpleDer(k3.getIzquierdo()));
    return rotacionSimpleIzq(k3);
}

private NodoBinario rotacionDobleDerIzq(NodoBinario k3) {
    k3.setDerecho(rotacionSimpleIzq(k3.getDerecho()));
    return rotacionSimpleDer(k3);
}
```

Fuente: elaboración propia.

Figura 5: Rotaciones dobles



Fuente: elaboración propia.

2. Balance

Una vez definidas las rotaciones, necesitamos un método que realice el balance cada vez que insertemos un elemento en el árbol. El siguiente código realiza esta operación utilizando el valor de las alturas de cada subárbol para ver cuál es el que se encuentra desbalanceado:

Figura 6: Código de balance

```java
private NodoBinario balancear(NodoBinario nodo) {
    if (nodo != null) {
        if (altura(nodo.getIzquierdo()) - altura(nodo.getDerecho()) == 2) {

            // Desequilibrio hacia la izquierda.
            int alturaIzqIzq = altura(nodo.getIzquierdo().getIzquierdo());
            int alturaIzqDer = altura(nodo.getIzquierdo().getDerecho());
            if (alturaIzqIzq >= alturaIzqDer) {

                // Desequilibrio simple hacia la izquierda.
                return rotacionSimpleIzq(nodo);
            } else {

                // Desequilibrio doble hacia la izquierda.
                return rotacionDobleIzqDer(nodo);
            }
        } else {
            if (altura(nodo.getDerecho()) - altura(nodo.getIzquierdo()) == 2) {

                // Desequilibrio hacia la derecha!
                int alturaDerDer = altura(nodo.getDerecho().getDerecho());
                int alturaDerIzq = altura(nodo.getDerecho().getIzquierdo());
                if (alturaDerDer >= alturaDerIzq) {

                    //desequilibrio simple hacia la derecha.
                    return rotacionSimpleDer(nodo);
                } else {

                    //desequilibrio doble hacia la derecha.
                    return rotacionDobleDerIzq(nodo);
                }
            }
        }
    }
    return nodo;
}
```

Fuente: elaboración propia.

El método realiza una comparación de las alturas del subárbol izquierdo con el subárbol derecho. Si la diferencia de alturas es igual a 2, entonces el árbol necesita ser balanceado por medio de rotaciones. Si la altura del subárbol izquierdo es mayor que la del subárbol derecho, entonces se sostiene que el desequilibrio está hacia la izquierda; en caso contrario, se afirma que el desequilibrio está hacia la derecha.

Una vez que sabemos de qué lado se encuentra el desequilibrio, debemos analizar si necesitamos una rotación simple o doble por medio de las alturas. Si es un desequilibrio hacia la izquierda, debemos conocer cuál de los subárboles del subárbol izquierdo tiene mayor altura. Si es el izquierdo, entonces se trata de una rotación simple o, si es el derecho, se trata de una rotación doble. El mismo razonamiento se aplica cuando es un desequilibrio hacia la derecha.

Insertar balanceado

Como se mostró anteriormente, cada vez que se modifique el árbol (por ejemplo, que se agreguen o eliminen elementos), corremos el riesgo de que pierda su propiedad de equilibrio en alguno de sus nodos, la cual debe conservarse si queremos obtener tiempos de ejecución de orden O(log(n)), en el peor de los casos.

La idea general que se utiliza en esta implementación de árboles AVL para implementar los algoritmos de inserción y de eliminación de nodos sobre un AVL es la siguiente:

- Efectuar los algoritmos de igual forma que en los árboles binarios de búsqueda, pero en cada recursión ir actualizando las alturas y rebalanceando el árbol en caso de que sea necesario.

Dado que ya vimos funciones para balancear un árbol, estamos listos para implementar el algoritmo de inserción:

Figura 7: Algoritmo de inserción

```
private NodoBinario insertar(Comparable x, NodoBinario nodo) {
    if (nodo == null) {
        nodo = new NodoBinario(x, null, null);
    } else if (x.compareTo(nodo.getDato()) < 0) {
        nodo.setIzquierdo(insertar(x, nodo.getIzquierdo()));
    } else if (x.compareTo(nodo.getDato()) > 0) {
        nodo.setDerecho(insertar(x, nodo.getDerecho()));
    }
    return balancear(nodo);
}
```

Fuente: elaboración propia.

Eliminar balanceado

La estrategia para diseñar el algoritmo de eliminación sobre árboles AVL es la misma que para la inserción:

- Se utiliza el mismo algoritmo que sobre árboles binarios de búsqueda, pero en cada recursión se detectan y corrigen errores por medio del método balancea.

La implementación en código Java es la siguiente:

Figura 8: Implementación en código Java

```java
private NodoBinario eliminar(Comparable x, NodoBinario nodo){
    if (nodo != null) {
        if (x.compareTo(nodo.getDato()) < 0) {
            nodo.setIzquierdo(eliminar(x, nodo.getIzquierdo()));
        } else {
            if (x.compareTo(nodo.getDato()) > 0){
                nodo.setDerecho(eliminar(x, nodo.getDerecho()));
            } else  {
                if (nodo.getIzquierdo() != null && nodo.getDerecho() != null) {
                    nodo.setDato(buscarMinimo(nodo.getDerecho()).getDato());
                    nodo.setDerecho(eliminarMinimo(nodo.getDerecho()));
                } else {
                    nodo = (nodo.getIzquierdo() != null) ? nodo.getIzquierdo() : nodo.getDerecho();
                }
            }
        }
    }
    return balancear(nodo);
}
```

Fuente: elaboración propia.

Recordamos un poco la idea del algoritmo de eliminación sobre árboles binarios de búsqueda. Primero se recorre el árbol para detectar el nodo que eliminar. Luego, encontramos tres casos que debemos diferenciar por su complejidad:

- Si dicho nodo es una hoja, procedemos a eliminarlo de inmediato, sin más.
- Si dicho nodo tiene un solo hijo, el nodo puede eliminarse después de ajustar un apuntador del padre para saltar el nodo.
- Si dicho nodo tiene dos hijos, el caso es un poco más complicado. Lo que se estila hacer (y que de hecho se hace en el algoritmo gracias a la función auxiliar eliminarMinimo) es reemplazar el nodo actual por el menor nodo de su subárbol derecho (y luego eliminar este).

3. Consideraciones sobre la altura

Como vimos anteriormente, se necesitará tener acceso a la altura de cada nodo del árbol

“Dado que la función para hallar la altura de un nodo dado en un árbol tiene un tiempo de ejecución de O(log(n)) en el peor caso” (Gurin, 2004, p. 6).

Esto nos impacta cada vez que pretendemos insertar o eliminar un nodo. Para solucionar este problema, podemos almacenar una variable altura en cada nodo e irla actualizando en las inserciones y eliminaciones que se efectúen sobre el árbol, y así conseguir que el método de la altura sea constante O(1).

De esta forma, podemos redefinir la función de altura de la siguiente manera, aprovechando el campo altura que ahora tiene cada nodo del árbol:

Figura 9: Función altura

```
private int altura(NodoBinario nodo) {
    if (nodo == null) {
        return -1;
    }
    return nodo.getAltura();
}
```

Fuente: elaboración propia.

Para soportar este cambio, modificamos la clase NodoBinario de la siguiente forma:

Figura 10: Modificación clase NodoBinario

```
private int altura;

public int getAltura() {
    return altura;
}

public void actualizarAltura() {
    int alturaIzq = 0;
    int alturaDer = 0;
    if (derecho != null) {
        alturaDer = derecho.getAltura();
    }
    if (izquierdo != null) {
        alturaIzq = izquierdo.getAltura();
    }
    this.altura = Math.max(alturaIzq, alturaDer) + 1;
}
```

Fuente: elaboración propia.

Por último, necesitamos llamar a la función actualizar Altura cada vez que insertemos o eliminamos un nodo para que la variable interna de la altura quede actualizada.

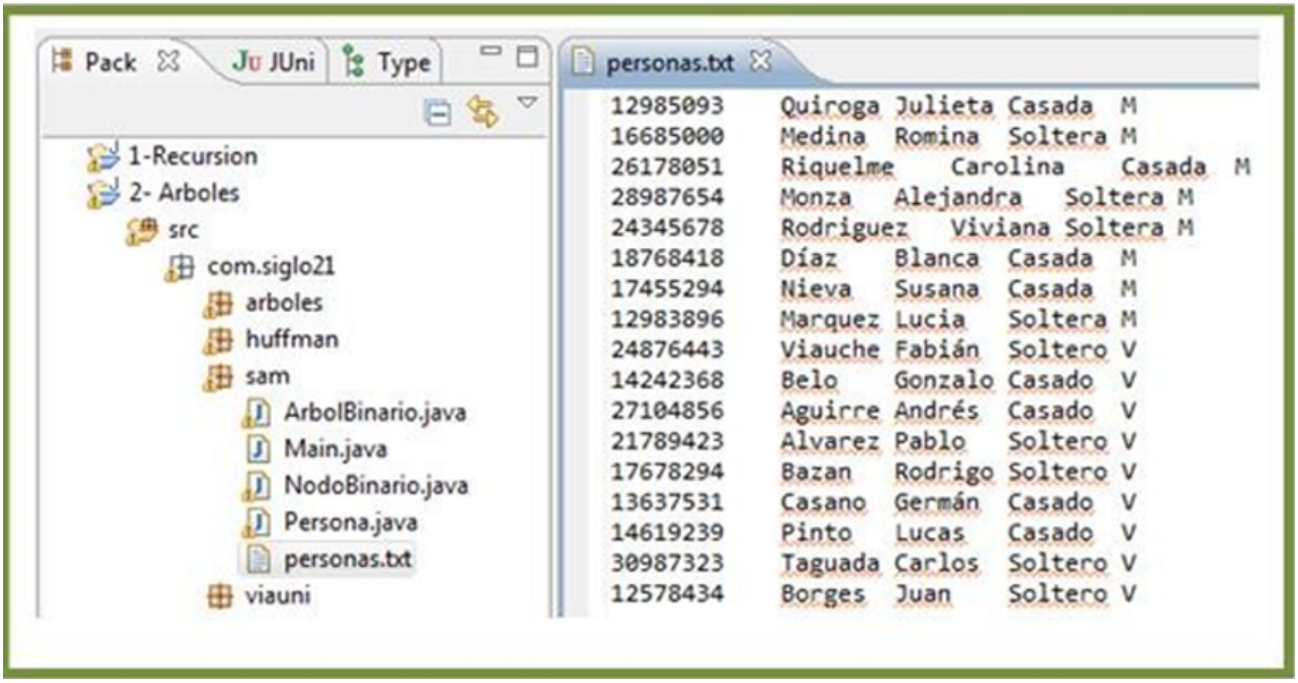
Apéndice A

En este apéndice, se muestran los pasos requeridos para obtener los datos desde un archivo de texto y facilitar las cargas de datos en las estructuras de datos.

Por ejemplo, dado un archivo de personas llamado “personas.txt”, en donde cada línea

del archivo tiene la información de 1 persona con cada dato separado por el tabulador, como se muestra a continuación:

Figura 11: Apéndice A



Fuente: elaboración propia.

En el siguiente código, se muestra cómo recuperar la información, cargarla en un objeto e insertarla en un árbol.

El parámetro fileName indica el nombre y la ruta en donde se encuentra el archivo. En este caso, el valor de la variable es:

“src/com/siglo21/sam/personas.txt”

Una vez creado el objeto del tipo FileInputStream, se crea el objeto DataInputStream para poder crear un objeto del tipo BufferReader. Así, podemos leer línea por línea el archivo y usar un *string* para separar los datos por medio del método *spllit*(‘\t’), que retorna un arreglo de los *strings* que están separados por el tabulador. Por último, se crea el objeto Persona y se inserta en el árbol.

Figura 12: Código Apéndice A

```java
private static ArbolBinario cargarPersonas(String fileName)
        throws NumberFormatException, IOException {
    FileInputStream fstream = new FileInputStream(fileName);
    DataInputStream dis = new DataInputStream(fstream);
    BufferedReader br = new BufferedReader(new InputStreamReader(dis));

    String line = "";
    ArbolBinario arbol = new ArbolBinario();
    while ((line = br.readLine()) != null && line.length() != 0) {
        String[] datos = line.split("\t");
        Persona persona = new Persona(
                Integer.valueOf(datos[0]), datos[1], datos[2], datos[3], datos[4]);
        arbol.insertar(persona);
    }
    return arbol;
}
```

Fuente: elaboración propia.

Referencias

Cormen, T., Leiserson, C., Rivest, R. y Stein, C. (2002). *Introduction to Algorithms* (2nd ed.). New York: McGraw Hill.

Gurin, S. (2004). *Árboles AVL*. Recuperado de http://es.tldp.org/Tutoriales/doc-programacion-arboles-avl/avl-trees.pdf

Weiss, M. A. (2000). *Estructuras de Datos en Java*. Buenos Aires, AR: Addison Wesley.